

**Data structures CCCS 224**

**Project 1**

**Coffee Shop Management System**

**Section: E3**

Enas Hamed AlQarni 2314104

Jana Akkad 2313200

Abrar Alharbi 2317129

## **Coffee Shop Management System**

The **Coffee Shop Management System** is a Java-based console application designed to streamline the management of a small coffee shop's operations. Built using object-oriented principles in NetBeans, the app handles customer orders, prioritizes urgent requests, tracks inventory, and manages order statuses efficiently.

### **Key Features:**

#### **1. Order Placement**

Customers can place regular or priority orders by specifying:

- Order ID
- Customer ID
- Name
- Selected beverage (from the menu)

#### **2. Order Queue Management**

The system supports:

- Priority and regular order queues
- Fulfilling the next order based on priority
- Canceling orders by Order ID
- Tracking order status: Pending, Completed, Cancelled

#### **3. Inventory Management**

- Initial inventory includes popular drinks (e.g., Latte, Espresso, Mocha, etc.), each with a quantity of 10.
- Automatic stock deduction upon order placement
- Out-of-stock detection
- Persistent inventory updates using file storage (stock.txt)

#### **4. Order Status Views**

Users can view:

- All pending (regular and priority) orders
- Completed orders
- Cancelled orders

#### **5. Data Persistence**

- All orders and inventory data are saved to files (orders.txt and stock.txt)
- On app startup, it automatically loads previous data if available

#### **6. User-Friendly Console Menu**

A simple numbered menu guides users to:

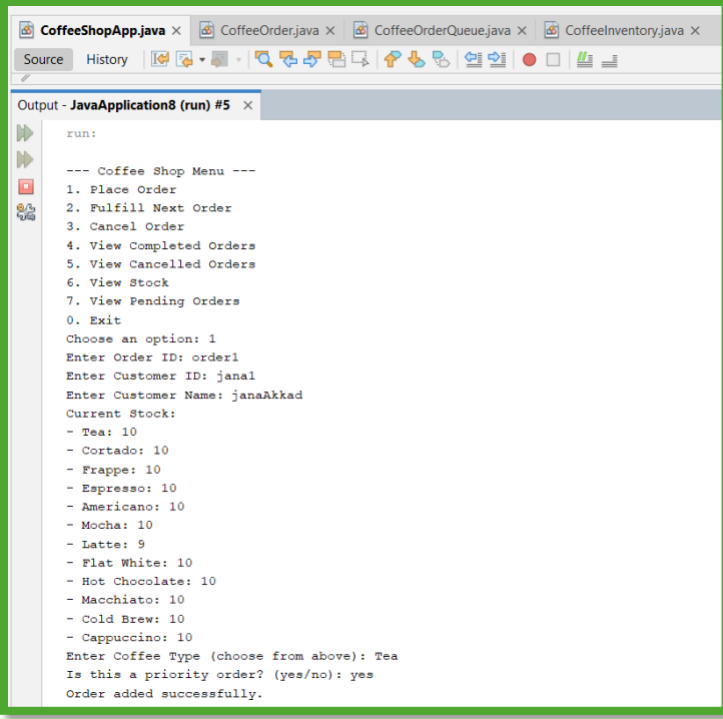
- Place or cancel orders
- Fulfill the next order
- Check stock levels and order history
- Exit the system safely while saving progress

## Placing an Order:

In this test run, the user selected the **“Place Order”** option from the main menu, which is **Option 1**.

1. The system asked for:
  - Order ID (order1)
  - Customer ID (jana1)
  - Customer Name (jana Akkad)
2. It then showed the available drinks and their stock levels.
3. The user chose **Tea** and marked the order as **priority**.
4. The system checked the stock, confirmed availability, deducted 1 from Tea's stock, and saved the changes.
5. The order was added to the **priority queue**, and a success message was shown.

This example shows how the system handles new orders while updating and saving inventory automatically.



```
Output - JavaApplication8 (run) #5 x
run:
--- Coffee Shop Menu ---
1. Place Order
2. Fulfill Next Order
3. Cancel Order
4. View Completed Orders
5. View Cancelled Orders
6. View Stock
7. View Pending Orders
0. Exit
Choose an option: 1
Enter Order ID: order1
Enter Customer ID: jana1
Enter Customer Name: janaAkkad
Current Stock:
- Tea: 10
- Cortado: 10
- Frappe: 10
- Espresso: 10
- Americano: 10
- Mocha: 10
- Latte: 9
- Flat White: 10
- Hot Chocolate: 10
- Macchiato: 10
- Cold Brew: 10
- Cappuccino: 10
Enter Coffee Type (choose from above): Tea
Is this a priority order? (yes/no): yes
Order added successfully.
```

## Viewing, Fulfilling, and Confirming an Order

1. The user selected **Option 7: View Pending Orders.**

The system displayed the current **pending priority orders**, showing:

- Order ID: order1
- Customer: jana Akkad
- Drink: Tea
- Status: Pending

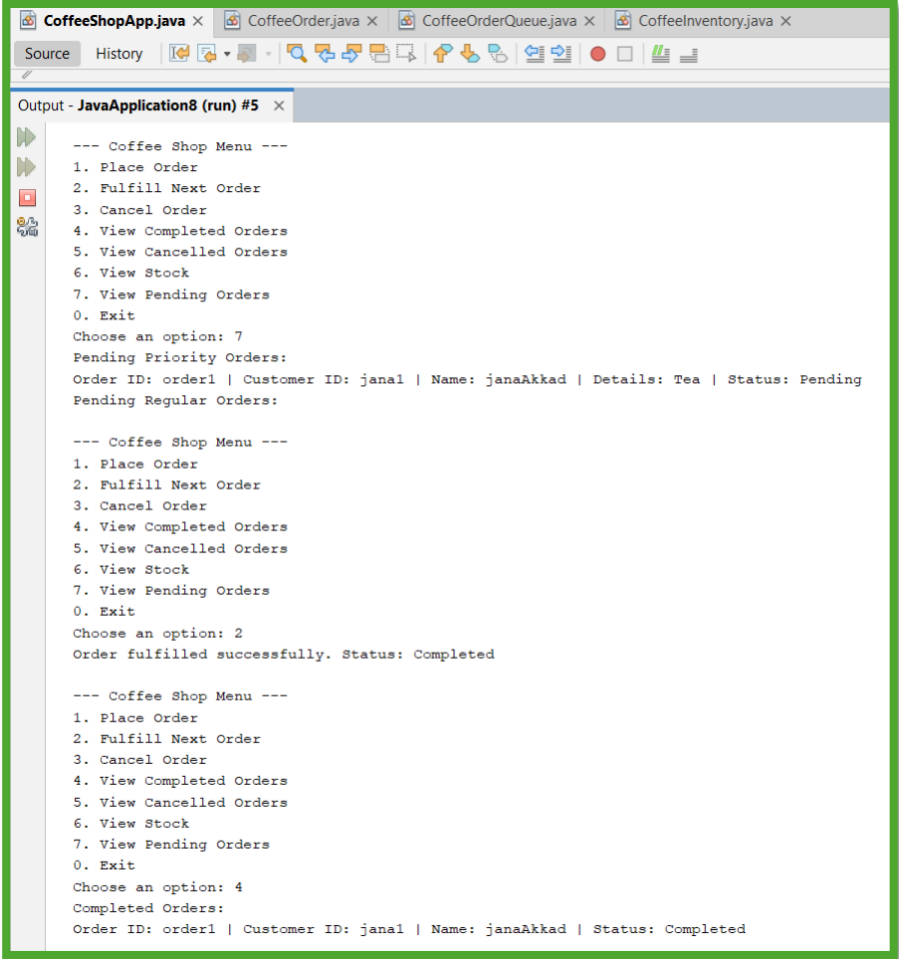
2. The user then chose **Option 2: Fulfill Next Order.**

Since order1 was the first in the priority queue, the system marked it as **Completed** and printed a confirmation message.

3. Next, the user selected **Option 4: View Completed Orders.**

The system displayed the details of order1, now updated with the **status "Completed"**.

This sequence demonstrates how the system manages order fulfillment: checking pending queues, updating order status, and showing results accurately in each view.



```
--- Coffee Shop Menu ---
1. Place Order
2. Fulfill Next Order
3. Cancel Order
4. View Completed Orders
5. View Cancelled Orders
6. View Stock
7. View Pending Orders
0. Exit
Choose an option: 7
Pending Priority Orders:
Order ID: order1 | Customer ID: jana1 | Name: janaAkkad | Details: Tea | Status: Pending
Pending Regular Orders:

--- Coffee Shop Menu ---
1. Place Order
2. Fulfill Next Order
3. Cancel Order
4. View Completed Orders
5. View Cancelled Orders
6. View Stock
7. View Pending Orders
0. Exit
Choose an option: 2
Order fulfilled successfully. Status: Completed

--- Coffee Shop Menu ---
1. Place Order
2. Fulfill Next Order
3. Cancel Order
4. View Completed Orders
5. View Cancelled Orders
6. View Stock
7. View Pending Orders
0. Exit
Choose an option: 4
Completed Orders:
Order ID: order1 | Customer ID: jana1 | Name: janaAkkad | Status: Completed
```

## Viewing Stock

The user selected **Option 6: View Stock** from the main menu.

The system displayed the current inventory levels for all available beverages. Each drink shows the remaining quantity in stock.

From the output, we can see that:

- **Tea** and **Latte** now have **9 units** each, meaning they were previously ordered and deducted.
- All other drinks remain at their default stock of **10 units**.

This confirms that the inventory system is correctly tracking and updating item quantities as orders are placed and fulfilled.

```
--- Coffee Shop Menu ---
1. Place Order
2. Fulfill Next Order
3. Cancel Order
4. View Completed Orders
5. View Cancelled Orders
6. View Stock
7. View Pending Orders
0. Exit
Choose an option: 6
Current Stock:
- Tea: 9
- Cortado: 10
- Frappe: 10
- Espresso: 10
- Americano: 10
- Mocha: 10
- Latte: 9
- Flat White: 10
- Hot Chocolate: 10
- Macchiato: 10
- Cold Brew: 10
- Cappuccino: 10
```

## Adding and Cancelling a Regular Order

1. The user began by placing a **new order** using **Option 1: Place Order**.

The following details were entered:

- Order ID: order2
- Customer ID: enas1
- Customer Name: Enas
- Selected Drink: Cold Brew
- Marked as **non-priority** (no)

Since Cold Brew was available in stock, the system:

- Accepted the order
- Deducted 1 from the Cold Brew stock
- Saved the changes and confirmed the order was added successfully

2. The user then chose **Option 3: Cancel Order** and entered the ID order2.

The system located the regular order and cancelled it, updating its status to **Cancelled**.

3. Using **Option 5: View Cancelled Orders**, the user verified that the order now appears under cancelled orders with the correct customer details and status.

This run shows how the system handles **non-priority (regular)** orders and allows for **cancellation** after placement. It also confirms that order status updates are properly reflected when viewing cancelled records.

```
Enter Order ID: order2
Enter Customer ID: enas1
Enter Customer Name: enas
Current Stock:
- Tea: 9
- Cortado: 10
- Frappe: 10
- Espresso: 10
- Americano: 10
- Mocha: 10
- Latte: 9
- Flat White: 10
- Hot Chocolate: 10
- Macchiato: 10
- Cold Brew: 10
- Cappuccino: 10
Enter Coffee Type (choose from above): Cold Brew
Is this a priority order? (yes/no): no
Order added successfully.

--- Coffee Shop Menu ---
1. Place Order
2. Fulfill Next Order
3. Cancel Order
4. View Completed Orders
5. View Cancelled Orders
6. View Stock
7. View Pending Orders
0. Exit
Choose an option: 3
Enter Order ID to cancel: order2
Order cancelled successfully. Status: Cancelled
```

```
--- Coffee Shop Menu ---
1. Place Order
2. Fulfill Next Order
3. Cancel Order
4. View Completed Orders
5. View Cancelled Orders
6. View Stock
7. View Pending Orders
0. Exit
Choose an option: 5
Cancelled Orders:
Order ID: order2 | Customer ID: enas1 | Name: enas | Status: Cancelled

--- Coffee Shop Menu ---
1. Place Order
2. Fulfill Next Order
3. Cancel Order
4. View Completed Orders
5. View Cancelled Orders
6. View Stock
7. View Pending Orders
0. Exit
Choose an option: |
```